DevOps Configuration Management Tools

Presented By:

2251187 Ankita Nath

2251188 Arpita Nath

CSEN3293 Term Paper

CSE – A,B

1. Introduction to DevOps

What is DevOps?

DevOps is a software development approach that combines development (Dev) and operations (Ops) to improve collaboration, efficiency, and automation in the software development lifecycle (SDLC).

Key DevOps Principles:

- **Automation**

Automation in DevOps removes **manual intervention**, reducing errors and speeding up processes.

Infrastructure as Code (IaC) → Automates server provisioning (Terraform, Ansible).
Configuration Management → Automates software installation and updates (Puppet, Chef).
Automated Testing → Runs tests automatically after code changes (Selenium, JUnit).
Monitoring & Alerts → Tracks system performance and errors (Prometheus, Grafana).

Example:

Google automates deployments using Kubernetes, reducing downtime.

- **Collaboration**

DevOps promotes **communication and teamwork** between developers, testers, and IT operations.

Uses **chat tools, dashboards, and shared repositories** to improve coordination.
Cross-functional teams help in troubleshooting issues faster.
Encourages **agile methodologies** like Scrum and Kanban.

Example:

Facebook adopts DevOps to ensure that developers and IT teams work together for daily software updates.

- **Continuous Integration & Continuous Deployment (CI/CD)**

CI/CD pipelines automate the software delivery process:

Continuous Integration (CI) → Developers frequently merge code into a shared repository (GitHub Actions, Jenkins).
Continuous Deployment (CD) → The system automatically deploys changes to production after passing tests (Docker, Kubernetes).

Reduces release cycles from weeks/months to **minutes/hours**.
Ensures quick rollback in case of failures.

Example:

Microsoft Azure DevOps enables CI/CD to roll out updates without downtime.

Enhances the Software Development Lifecycle (SDLC)

How DevOps Improves SDLC?

The **Software Development Lifecycle (SDLC)** includes:

1. **Planning** → Define requirements & project goals.
2. **Development** → Write and test code.
3. **Build & Integration** → Merge code into a shared repository.
4. **Testing** → Run automated/manual tests to detect bugs.
5. **Deployment** → Deploy to production using CI/CD pipelines.
6. **Monitoring & Feedback** → Track performance and collect user feedback.

DevOps speeds up each stage through automation and collaboration.
Minimizes risks and improves software quality.

Example:

Spotify follows DevOps to continuously release new features without breaking the app.

Why is DevOps Important?

- Reduces deployment failures and rollbacks.
- Accelerates software delivery cycles.
- Improves collaboration between development and IT teams.
- Enhances scalability and flexibility.

2. Infrastructure as Code (IaC) vs. Configuration as Code (CaC)

Factor	Infrastructure as Code (IaC)	Configuration as Code (CaC)
Purpose	Manages infrastructure provisioning (VMs, networks, storage).	Manages software configurations on provisioned infrastructure.

Scope	Entire cloud/on-premise infrastructure.	Application and OS configurations.
Action	Creates, updates, and deletes infrastructure resources.	Defines how software, services, and applications should be configured.
Tools	Terraform, CloudFormation, Pulumi	Ansible, Chef, Puppet, SaltStack

Example:

- **IaC**: Terraform script to create an AWS EC2 instance.
- **CaC**: Ansible playbook to configure and install software on the instance.

3. What is a Configuration Management Tool?

A **Configuration Management (CM) tool** automates the **setup, management, and monitoring of software configurations** on servers and applications.

- **Ensures consistency** across environments.
- **Automates repetitive tasks** such as software installations and updates.
- **Tracks and manages changes** in configurations.

Examples: **Ansible, Chef, Puppet, SaltStack**

4. Importance of Configuration Management in DevOps

1. Version Control

- Configuration files are stored in **Git** for tracking changes and rollback if needed.
- Example: **Ansible playbooks stored in GitHub for easy collaboration.**

2. Infrastructure as Code (IaC)

- Automates infrastructure deployment, reducing manual intervention.
- Example: **Using Terraform to deploy VMs, then Ansible to configure them.**

3. Automated Provisioning

- Automatically sets up **servers, databases, and networks** using configuration scripts.
- Example: **Provisioning a Kubernetes cluster with Ansible.**

4. Application Configuration

- Ensures that applications are set up correctly across multiple environments.
- Example: **Configuring Nginx using Ansible.**

5. Automated Deployment

- Ensures seamless software deployments through CI/CD pipelines.
- Example: **Deploying a web application using Jenkins and Ansible.**

6. Environment Management

- Maintains identical configurations across **dev, staging, and production** environments.
- Example: **Using Puppet to standardize server configurations.**

5. Popular DevOps Configuration Management Tools

1. Ansible

Push-based model, no agent required.
Uses **YAML playbooks** for automation.
Best for **quick deployments and ad-hoc changes**.

2. Puppet

Pull-based model with an agent installed on nodes.
Uses **manifest files (`.pp`) written in Ruby-like DSL**.
Best for **large-scale, continuous infrastructure management**.

3. Chef

Pull-based model with an agent (Chef Client).
Uses **cookbooks and recipes** written in Ruby.
Best for **complex infrastructure automation**.

4. SaltStack

Supports both push and pull models.
Uses YAML for configuration and Python for scripting.
Best for **high-speed, large-scale automation**.

6. Configuration Management as Code (CMaC)

Pull-Based Configuration Management:

- **Tools:** Chef, Puppet
- **How It Works:**
  - Nodes **pull** configuration updates from a central server at regular intervals.
  - Ensures **continuous enforcement of configurations**.

Push-Based Configuration Management:

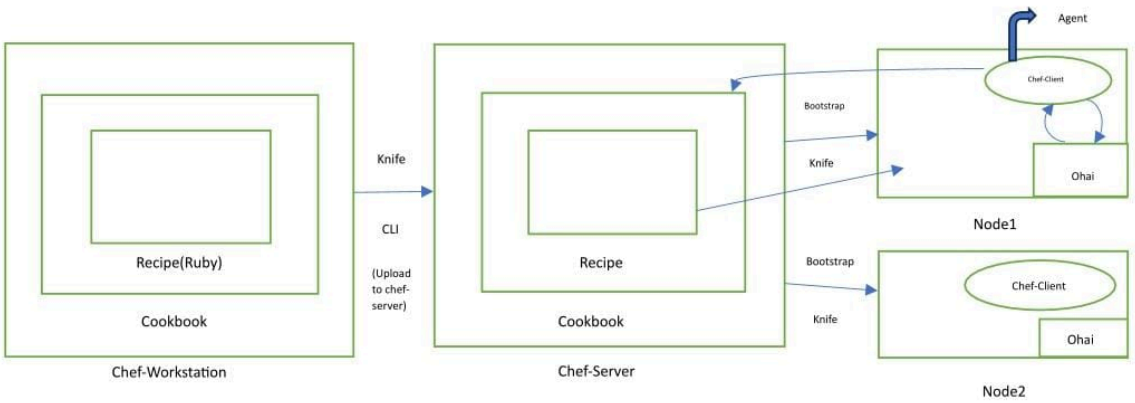
- **Tools:** Ansible, SaltStack
- **How It Works:**
  - The central control system **pushes** configurations to nodes in real time.
  - Ideal for **rapid changes and quick updates**.

Differences Between Pull-Based and Push-Based CM:

Feature	Push-Based CM	Pull-Based CM

Control Mechanism	Central server pushes updates	Nodes pull updates from a central server
Execution Mode	Instant, manual trigger	Automatic, periodic updates
Scalability	Best for small/medium environments	Best for large-scale infrastructure
Network Load	Higher due to active pushing	Lower, as nodes update at intervals
Failure Handling	Requires manual retry if push fails	Self-healing—nodes retry pulling updates
Example Tools	Ansible, SaltStack (Push Mode)	Puppet, Chef

7. Chef - Architecture or Process



CHEF-Architecture or Process

Chef Components:

1. **Chef Workstation** → Where you write code.
2. **Chef Server** → Where you upload code.All cookbooks are stored here.

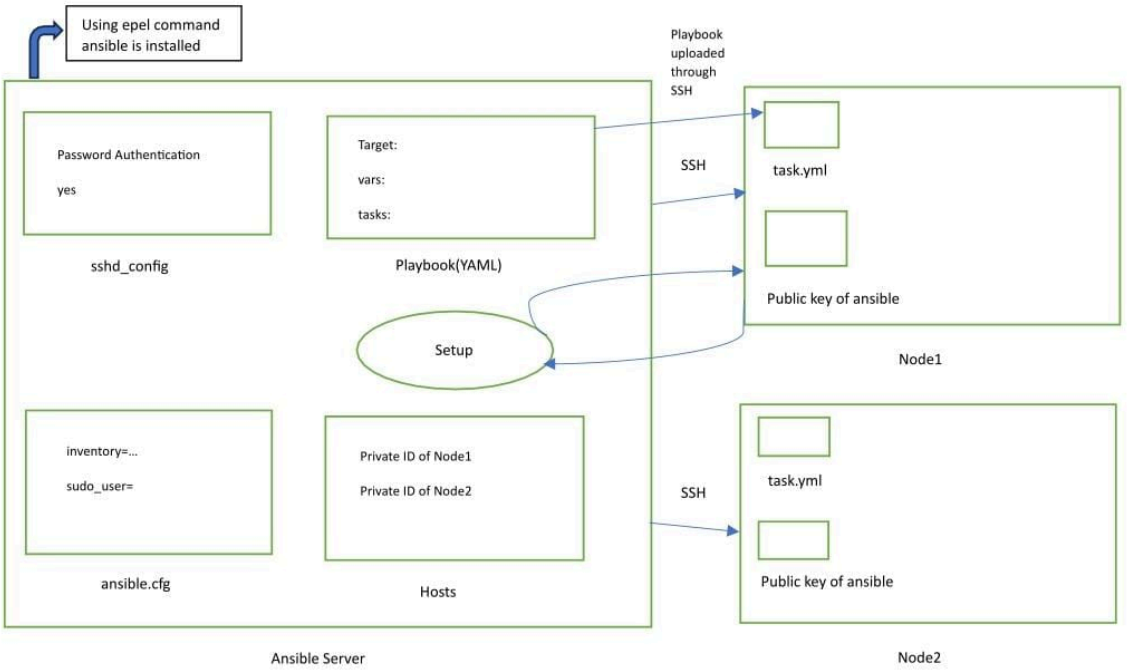
3. **Chef Node** → Where you apply code.
4. **Knife** → Tool to establish communication among workstation, server and node knife.Workstation communicate with the chef server using knife.Knife is a command-line tool that uploads the cookbook to the server.
5. **Chef-client** → Tool runs on every chef node to pull code/ recipe from chef-server.
6. **Ohai** → Maintain current state information of chef-node. Ohai is like a database of that node.
7. **Bootstrapping** → The process of attaching a node to chef-server.
8. **CLI** → Command Line Interface.
9. **Recipes** → Recipes are the codes and it is written in Ruby.
10. **Cookbook** → Collection of recipes. Cookbooks are stored in chef-server.
11. Ohai fetches the current state of the node it is located in.
12. Node communicate with the chef-server using the chef-client through knife(CLI Tool).
13. Chef-client is installed on every node.

How Chef Works?

Also known as Chef-Workstation, inside this we have cookbook, inside that we have recipe (written in Ruby language). Chef-workstation uploads the cookbook to the chef-server using the CLI-tools, Knife. Chef-server is connected to the nodes using CLI tool knife and the process is called “Bootstrapping”.Nodes pulls the recipe from the Chef-server.

On each node we have an Ohai, that acts as a database for each of the nodes. In Nodes we have an agent Chef-client, that goes to ohai, Ohai updates the chef-client. Chef-client goes to chef-server and it checks what is new that is to be updated, and pulled the new update to the nodes.

8. Ansible - Architecture or Process



ANSIBLE- Architecture or Process

Ansible Components:

1. Using shell command ansible is installed in the **Ansible server.**
2. YAML (**Yet Another Markup Language**).
3. Setup is like a database, it will go to the node and check if new updates are needed or not and from the nodes it gets back to ansible server to deliver that message.
4. Communication between **Ansible server and node** is done through **SSH.**
5. **Playbooks are divided into many sections like:**

   Target section → Define the host against which playbook task has to be executed.

   Variable section → Define variables.

   Task section → List of all modules that we need to run in an order.

6. **Other definitions:**

   Ansible server → The machine where Ansible is installed and from which all tasks and playbooks will be executed.

Modules → pre-built commands for performing tasks.

Managed node → Do not require Ansible installation.

sshd_config → The sshd config file controls SSH server settings (i.e., password authentication, root login, port change).

ansible.cfg → The ansible.cfg file is used to configure how Ansible behaves, including SSH settings, inventory location, privilege escalation, logging, etc.

How Ansible Works?

First, by using ‘epel’ command ansible is installed in ansible server where ‘ansible.cfg’, ‘sshd-config’ file is created automatically. ‘ansible.cfg’ is used to config how ansible behaves, ‘sshd-config’ file controls SSH server settings(password, authorization) to give permission for the access of node to the server and able to push updates to node.

There is a Host file that contains the private IDs of the nodes. Playbook is created inside the ansible server and it is written in “YAML language”. It consists of many sections like Target section, Variable section, Task section.

Ansible server is connected to the node through SSH. Playbook is updated to the node via SSH. These nodes contains the public key of ansible.

In ansible server there is setup. Setup is like a database, it will go to the node and check if new updates are needed or not and from the nodes it gets back to ansible server to deliver that message.

9. Use Cases of Push-Based & Pull-Based Configuration Management

Configuration Management (CM) tools play a crucial role in automating IT infrastructure management. Based on **push-based** and **pull-based** models, different tools serve different use cases. Below is a detailed breakdown of how these models are applied in real-world scenarios.

Push-Based Configuration Use Cases (Ansible, SaltStack)

Push-based CM tools operate by **actively sending configurations** from a central server to managed nodes. This is useful when immediate updates are required.

1.Emergency Security Patches & Critical Updates

Scenario:
A newly discovered **security vulnerability** requires an **immediate patch** across all servers in an enterprise.

How Push-Based CM Helps:

- The admin runs an **Ansible Playbook** to push security patches to all affected servers.
- **No agent installation is required**—the updates are sent via **SSH**.
- All servers **instantly receive and apply** the patch, ensuring security compliance.

Example:
Imagine a **Linux Kernel vulnerability (CVE-2024-XXXX)** is detected. Using Ansible, the security team can instantly deploy the patch across **hundreds of Linux servers** within minutes.

2. Rapid Deployments in DevOps (CI/CD Pipelines)

Scenario:
A company practicing **Continuous Integration/Continuous Deployment (CI/CD)** needs to **automate application deployments** with every code commit.

How Push-Based CM Helps:

- After a **successful code build**, Ansible triggers an **automated deployment** to production servers.
- Ansible pushes the latest **Docker container updates** or **application binaries**.
- **Zero downtime deployment** is achieved using rolling updates.

Example:
A fintech company releases new features **every week**. Using **Ansible + Jenkins**, every release is automatically **deployed across multiple servers**, reducing manual intervention.

3. Managing Small-to-Medium Scale Infrastructure

Scenario:
A startup with **50-200 servers** needs to **automate configurations** without setting up **pull-based agents**.

How Push-Based CM Helps:

- **No need for persistent agent installations**—Ansible connects via SSH.
- The **admin centrally defines** infrastructure configurations and pushes them instantly.
- Quick execution without maintaining a **centralized configuration repository**.

Example:
A growing e-commerce business with **100 virtual machines** wants to configure **database replication, firewall rules, and monitoring tools**. Ansible provides a **lightweight** solution without requiring a **dedicated configuration server**.

4. One-Time or Ad-Hoc Configuration Changes

Scenario:
A system admin needs to make a **one-time update**, such as **changing user permissions** or **modifying a configuration file** across multiple servers.

How Push-Based CM Helps:

- The admin **writes a simple playbook** and runs it **once** to apply the change.
- No persistent agents are needed—**the task is completed and the connection is closed**.

Example:
The IT team needs to **disable root login** via SSH on all servers. Using **Ansible**, they push the updated **sshd_config** file and restart the service across **hundreds of machines in seconds**.

Pull-Based Configuration Use Cases (Chef, Puppet)

Pull-based CM tools rely on **agents** installed on each node, which periodically pull configurations from a central server.

1. Large-Scale Cloud Infrastructure (AWS, Azure, GCP)

Scenario:
A multinational company manages **thousands of virtual machines** across **multiple cloud providers** and requires **automated, scalable configuration management**.

How Pull-Based CM Helps:

- Each node runs a **Puppet agent** or **Chef client**.

- The **centralized configuration server (Master)** contains all required configurations.
- Each node **periodically checks for updates** and applies them automatically.

Example:
A global **streaming service (like Netflix)** uses **Puppet** to maintain consistency across **100,000+ cloud servers** worldwide. New configuration changes **automatically propagate** to all instances.

2. Continuous Compliance & Security Policy Enforcement

Scenario:
A **finance or healthcare company** must enforce strict **compliance policies** like **PCI-DSS, HIPAA, or GDPR**.

How Pull-Based CM Helps:

- Nodes regularly **pull security configurations** from **Puppet or Chef Master**.
- The system automatically enforces **firewall rules, SSH policies, and access control lists**.
- If a node deviates from the expected configuration, it **auto-corrects itself**.

Example:
A bank uses **Chef** to enforce **security hardening policies** across **thousands of ATMs**, ensuring compliance with **PCI-DSS**. If an ATM configuration is changed **manually**, it **reverts back to the compliant state automatically**.

3. Auto-Scaling and Dynamic Infrastructure

Scenario:
A cloud-based SaaS platform **dynamically scales servers** based on user demand.

How Pull-Based CM Helps:

- **Newly provisioned servers** automatically install the latest configuration when they come online.
- There is no need for **manual intervention**—nodes self-configure.

Example:
A gaming company launches an **online multiplayer game** with **auto-scaling** based on peak traffic. When **new servers spin up**, they **automatically configure** themselves using **Puppet or Chef**, ensuring uniform settings.

4. Managing Remote and Occasionally Connected Servers

Scenario:
A **global retail chain** has **point-of-sale (POS) systems** in multiple locations. These systems may not always be connected to the corporate network.

How Pull-Based CM Helps:

- When the POS system **connects to the network**, it **checks for updates** and applies necessary changes.
- Ensures consistency **even if the device was offline for an extended period**.

Example:
A **fast-food chain** uses **Chef** to manage configurations across **thousands of POS systems**. Whenever a store's system **reconnects to the internet**, it **automatically updates** itself with the latest configurations.

Conclusion:

• Configuration management is key to DevOps success as it

1. Ensures consistency and standardization across environments.
2. Automates deployment and management of configurations.
3. Provides version control and auditing capabilities.
4. Improves collaboration and communication among teams.
5. Enhances security and compliance.

• Choosing the right tool depends on infrastructure needs:

Consider the following factors:

1. Infrastructure complexity: Larger, more complex infrastructures require more robust tools.
2. Scalability: Choose a tool that can scale with your growing infrastructure.
3. Security: Ensure the tool provides adequate security features.
4. Integration: Select a tool that integrates with your existing tools and workflows.
5. Learning curve: Consider the ease of use and learning curve for your team.

• Automation Reduces Time and Improves Efficiency:

Automation is a key aspect of configuration management. By automating repetitive tasks and workflows, you can:

1. Reduce manual errors.
2. Increase efficiency.
3. Decrease deployment time.
4. Improve consistency.
5. Enhance scalability.

Resources:

Official Documentation

Ansible → https://docs.ansible.com/

Chef → https://docs.chef.io/